UNIVERSITY INSTITUTE OF COMPUTING
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

MINI PROJECT REPORT

ON

"Himalayan Ride and Care: Motorcycle Rental and Roadside Assistance Management System"

Submitted in partial fulfilment of the requirements of

Advanced Internet Programming (AIP)

Submitted By:

Ayush Rana, UID 26MCI10020

Class/Section/Group:

_____

Under the Guidance of:

_____

University Institute of Computing

Chandigarh University, Mohali, Punjab

CU CHANDIGARH UNIVERSITY

UNIVERSITY INSTITUTE OF COMPUTING
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

CERTIFICATE

This is to certify that Ayush Rana (UID 26MCI10020), a student of the Master of Computer Applications programme, has prepared the project report titled “Himalayan Ride and Care: Motorcycle Rental and Roadside Assistance Management System” in partial fulfilment of the requirements of Advanced Internet Programming (AIP) at the University Institute of Computing, Chandigarh University, Mohali, Punjab.

The report presents the design, implementation and academic study of the Himalayan Ride and Care system. The project details and results are described for review by the project guide and department.

_____

Project Guide / Supervisor

University Institute of Computing

Date: _____

Place: Chandigarh University, Mohali, Punjab

ACKNOWLEDGEMENT

I thank my project guide and the faculty of the University Institute of Computing for their teaching and guidance. The Advanced Internet Programming course helped me connect browser forms, server-side Java, database persistence and HTTP services in one working application. I also acknowledge the case study and course materials that shaped the system requirements.

Date: _____

Place: Chandigarh University, Mohali, Punjab

Ayush Rana

UID 26MCI10020

ABSTRACT

Himalayan Ride and Care is a local web application for motorcycle rental operations and roadside support in Himalayan travel regions. It maintains customer and motorcycle information, records rental requests, tracks pickup and return odometer readings, stores payment entries and follows assistance cases from opening through resolution.

The browser interface is built with HTML, CSS and JavaScript. It exchanges JSON with a Java Spring Boot 3.3.5 server over REST-style HTTP endpoints. Embedded Tomcat accepts the requests; Spring Data JPA and Hibernate map application entities to MySQL 8.0 tables, while JdbcTemplate executes the dashboard summary query. The run record confirms Java 21, a successful MySQL connection and the web server on port 8080.

This report relates the implementation to the AIP syllabus topics of client-server communication, HTTP request and response handling, JDBC, ORM, Spring Boot, REST methods and JSON. The current application does not separately implement classic JSP pages, explicit HttpSession workflows, XML responses, or every HTTP verb in the lab syllabus; those distinctions are identified for study.

TABLE OF CONTENTS

1. Introduction and Objectives 0

2. Requirements and Scope 0

3. Design 0

4. Implementation 0

5. Testing, Results and Outputs 0

6. Conclusion and Limitations 0

References 0

1. Introduction and Objectives

A motorcycle-rental operator needs a current view of its fleet, reservations, returns, payment records and support requests. When these details are kept in separate registers, a staff member may miss a maintenance status or a conflicting booking, and trip information can be difficult to find when a rider asks for assistance. Himalayan Ride and Care brings these records into one browser-based application backed by a relational database.

The project applies Internet programming concepts by letting a browser submit HTTP requests to a Java server. The server validates input, coordinates the workflow and reads or writes records in MySQL. It is an academic local prototype, not a deployed rental service.

Project objectives:

1. Maintain customer, motorcycle, destination, rental, payment and roadside-assistance records.
2. Make fleet and rental status visible from one dashboard.
3. Reject invalid dates, unavailable motorcycles, overlapping rental periods and inconsistent odometer values.
4. Expose browser operations through HTTP endpoints that exchange JSON.
5. Demonstrate the relationship among Spring Boot controllers, Hibernate/JPA entities, repositories, JDBC and MySQL.

2. Requirements and Scope

The case study describes tourists, rental staff, fleet managers, support operators and business managers. The current prototype implements the shared operational tasks through a local interface; it does not authenticate users or enforce separate staff roles.

User or stakeholder	Information or task supported
Tourist / customer	Contact record, rental request and roadside issue details.
Rental desk staff	Create and confirm requests; record pickup and return readings; enter payment records.
Fleet manager	Review motorcycles and change availability, maintenance or inactive status.
Support operator	Open an assistance case, assign a responder and update case progress.
Business manager	Review current fleet, rental and open-support counts.

Functional scope includes customer and motorcycle creation, destination display, rental status changes, checkout and return, payment records, assistance tracking, and a dashboard summary. The application validates key business rules on the server. It has no login, authorization roles, live GPS, map or weather service, payment gateway, insurance workflow, e-mail notifications or automatic emergency dispatch.

3. Design

3.1 Application architecture

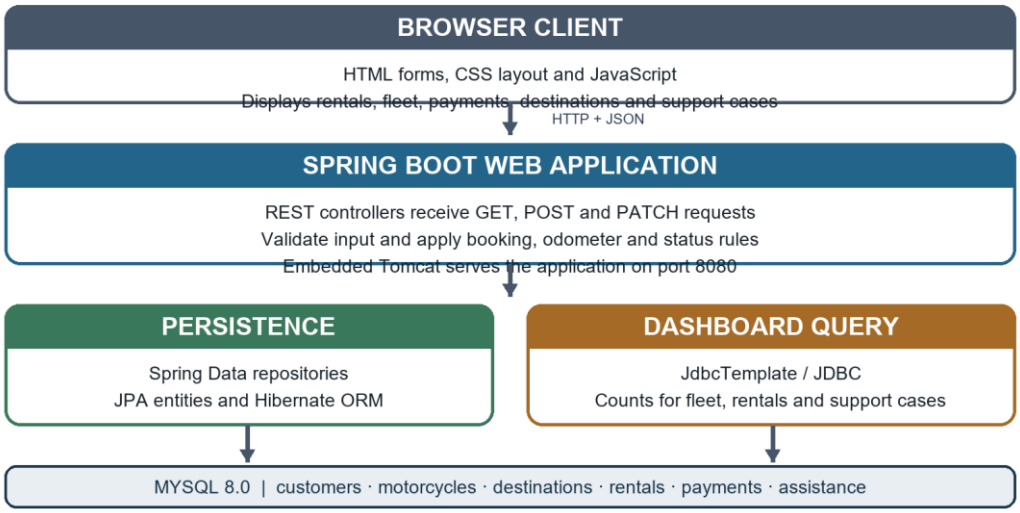


Figure 1: Browser, Spring Boot, persistence and MySQL layers

The browser sends HTTP requests and JSON to controllers in the Spring Boot application. Controllers validate request data and coordinate the case-study operations. Spring Data repositories use JPA and Hibernate for entity persistence. The dashboard's summary values are read through JdbcTemplate. The database stores the six core case-study relations. The application runs with embedded Tomcat, so a separate Tomcat installation is not needed.

3.2 Data model

Relation	Purpose and links
customers	Customer name, phone and optional unique e-mail; referenced by rentals and support cases.
motorcycles	Unique registration, model, daily rate, odometer and operational status.
destinations	Destination name and description; optionally selected for a rental.
rental_bookings	Links customer and motorcycle, optional destination, rental dates, status and odometer readings.
payment_records	One record per payment entry, linked to a rental booking.
assistance_requests	Customer, optional booking, issue type, priority, location, responder and status.

A customer can have many bookings and assistance requests. A motorcycle can be rented many times over its lifetime; the service checks that active, confirmed or pending periods do not overlap. A booking can have several payment entries. A roadside request must identify a customer and may optionally point to a booking.

3.3 Rental and support workflows

A rental moves from PENDING to CONFIRMED, ACTIVE and COMPLETED; it may be CANCELLED before checkout. Checkout stores the pickup odometer. Return stores the final odometer and updates the motorcycle reading. Assistance cases move through OPEN, ASSIGNED, IN_PROGRESS, RESOLVED and CLOSED, with CANCELLED also available. These states describe records in the application; an URGENT assistance case does not contact emergency services.

4. Implementation

4.1 REST endpoints and HTTP methods

The controllers use GET to read records, POST to create records or perform checkout and return actions, and PATCH to change selected status or assignment fields. A browser form sends the POST request; typing a path in the address bar sends a GET request.

Method	Example path	Purpose
GET	/api/customers	List customers.
POST	/api/customers	Create a customer from JSON.
GET	/api/rentals	List rental bookings.
POST	/api/rentals	Create a pending rental request.
PATCH	/api/rentals/{id}/status	Confirm or cancel a request.
POST	/api/rentals/{id}/checkout	Record pickup and activate a booking.
POST	/api/rentals/{id}/return	Record return and complete a booking.
GET / POST / PATCH	/api/assistance	List, open and update assistance cases.

Example JSON body for a rental request:

```
{
  "customerId": 1,
  "motorcycleId": 1,
  "destinationId": 1,
  "startDate": "2027-06-10",
  "endDate": "2027-06-14"
}
```

4.2 Persistence and validation

Java model classes marked as JPA entities represent customers, motorcycles, destinations, rentals, payments and assistance requests. Repository interfaces provide common CRUD operations. Hibernate maps entity fields and relationships to MySQL; the project configuration uses schema update during local startup. JdbcTemplate executes the dashboard summary query. The MySQL Connector/J driver supplies the Java database connection.

Server-side rules check the requested rental period against existing pending, confirmed and active bookings. A motorcycle in maintenance or inactive status cannot be newly rented. Checkout cannot use an odometer below the motorcycle's current reading, and return cannot be lower than the pickup reading. Payment rows are records only and do not charge a payment provider.

4.3 AIP syllabus alignment

AIP topic	Use in this project
Client-server web application	Browser interface calls a Java server over HTTP.
Servlet container and request/response	Spring Boot uses embedded Tomcat to accept HTTP requests and return responses.
JDBC and CRUD	JPA repositories handle the main records; JdbcTemplate performs the dashboard query; MySQL Connector/J provides JDBC connectivity.
Hibernate ORM	JPA entities map Java objects and relationships to database tables.
Spring Framework and Spring Boot	Controllers, validation, repositories and embedded server are wired through Spring Boot.
REST, HTTP methods and JSON	GET, POST and PATCH endpoints exchange JSON with the browser.
JSP, HttpSession, XML, PUT and DELETE	Not separate implementations in this version; compare these syllabus topics with the project's static HTML/JavaScript UI and JSON API.

5. Testing, Results and Outputs

The application was launched locally from Eclipse with Java 21.0.13. The startup output records six JPA repository interfaces, a successful Hikari/MySQL connection and embedded Tomcat listening on port 8080. The project schema and sample records were created in MySQL Workbench. Operational test entries were later deleted successfully from payment_records, assistance_requests and rental_bookings. No automated JUnit test suite or performance benchmark is included.

Manual check	Observed result
Project compilation and startup	Spring Boot startup completed in Eclipse.
Database connection	Connection pool established a MySQL connection.
Dashboard and sample data	The browser app reads customers, motorcycles and destinations from the database.
Operational record cleanup	Workbench deleted 10 payment rows, 3 assistance rows and 6 rental rows; related tables were left available for new trials.

5.1 Results and application outputs

The user-provided screenshots were captured from the running application. They show five available motorcycles, one current rental and one open assistance case; the fleet records include registration, model, daily rate, odometer and status. The rental row is confirmed for Asha Sharma and Manali. The final screen shows the payment and roadside-support forms; the screenshot displays a recorded transaction line as well. These interface captures demonstrate visible outputs and do not claim that every form shown has just been submitted.

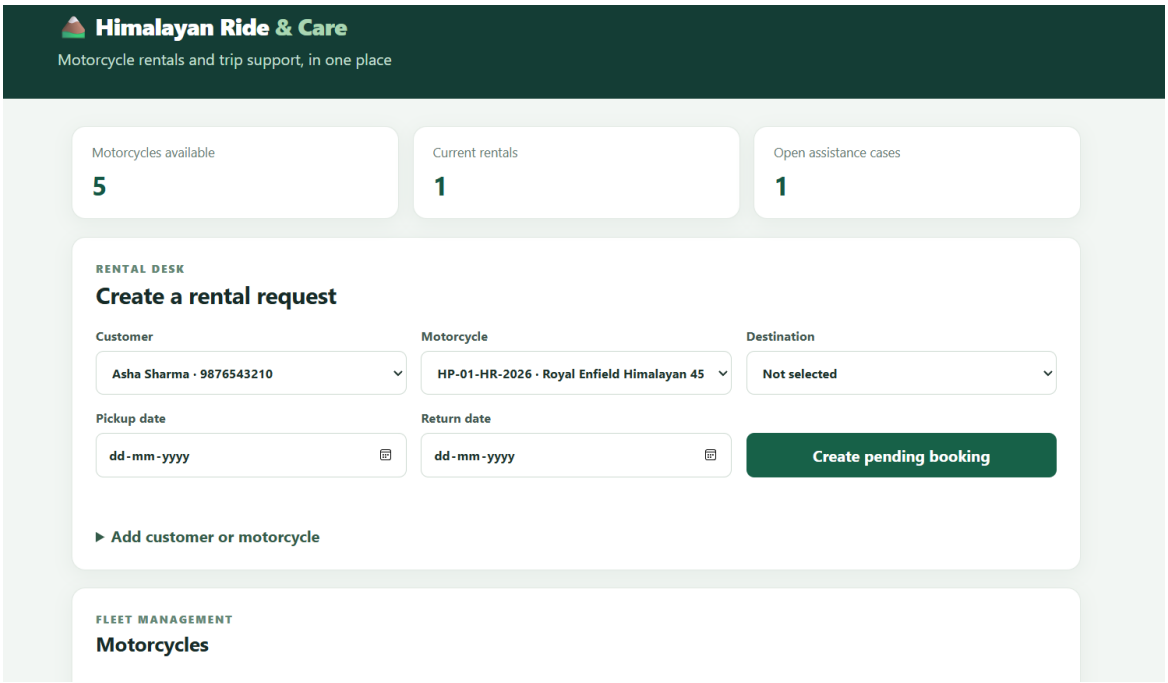
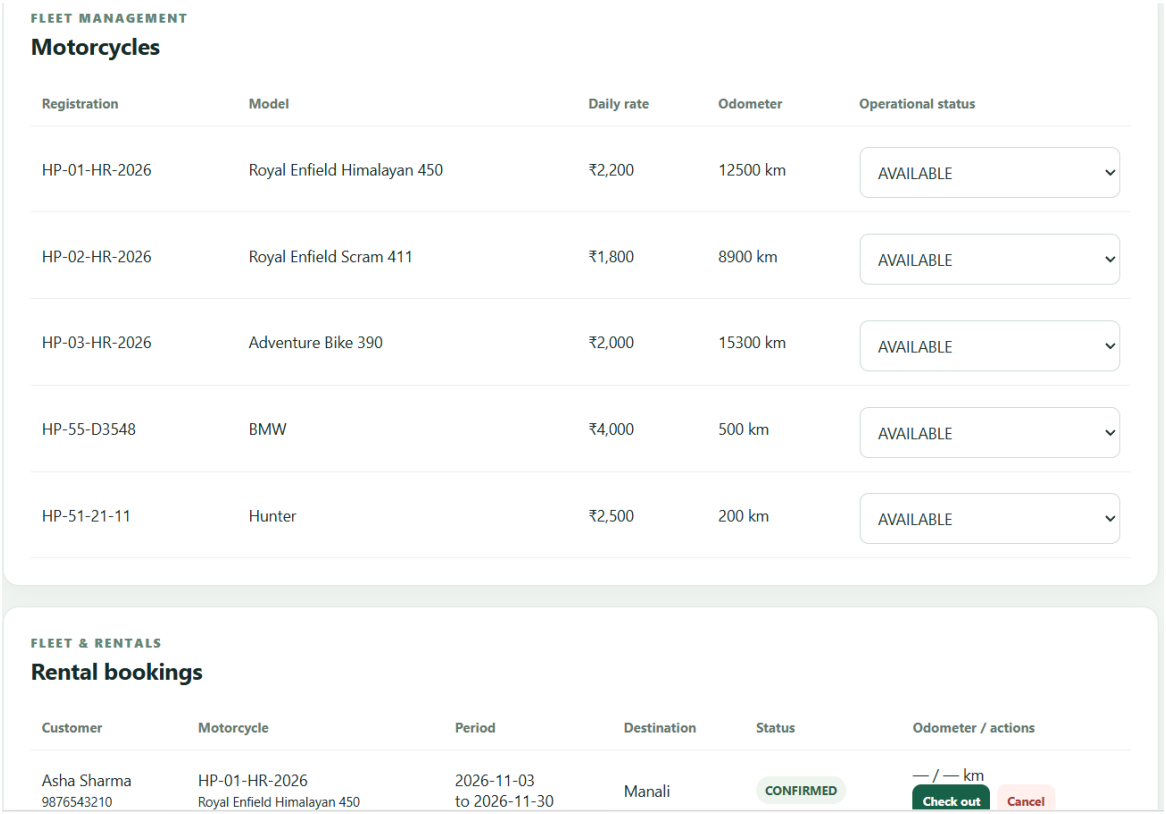


Figure 2: Dashboard summary and rental request form in Himalayan Ride and Care.



FLEET MANAGEMENT

Motorcycles

Registration	Model	Daily rate	Odometer	Operational status
HP-01-HR-2026	Royal Enfield Himalayan 450	₹2,200	12500 km	AVAILABLE
HP-02-HR-2026	Royal Enfield Scram 411	₹1,800	8900 km	AVAILABLE
HP-03-HR-2026	Adventure Bike 390	₹2,000	15300 km	AVAILABLE
HP-55-D3548	BMW	₹4,000	500 km	AVAILABLE
HP-51-21-11	Hunter	₹2,500	200 km	AVAILABLE

FLEET & RENTALS

Rental bookings

Customer	Motorcycle	Period	Destination	Status	Odometer / actions
Asha Sharma 9876543210	HP-01-HR-2026 Royal Enfield Himalayan 450	2026-11-03 to 2026-11-30	Manali	CONFIRMED	— / — km Check out Cancel

Figure 3: Fleet management output and the start of the rental bookings list.

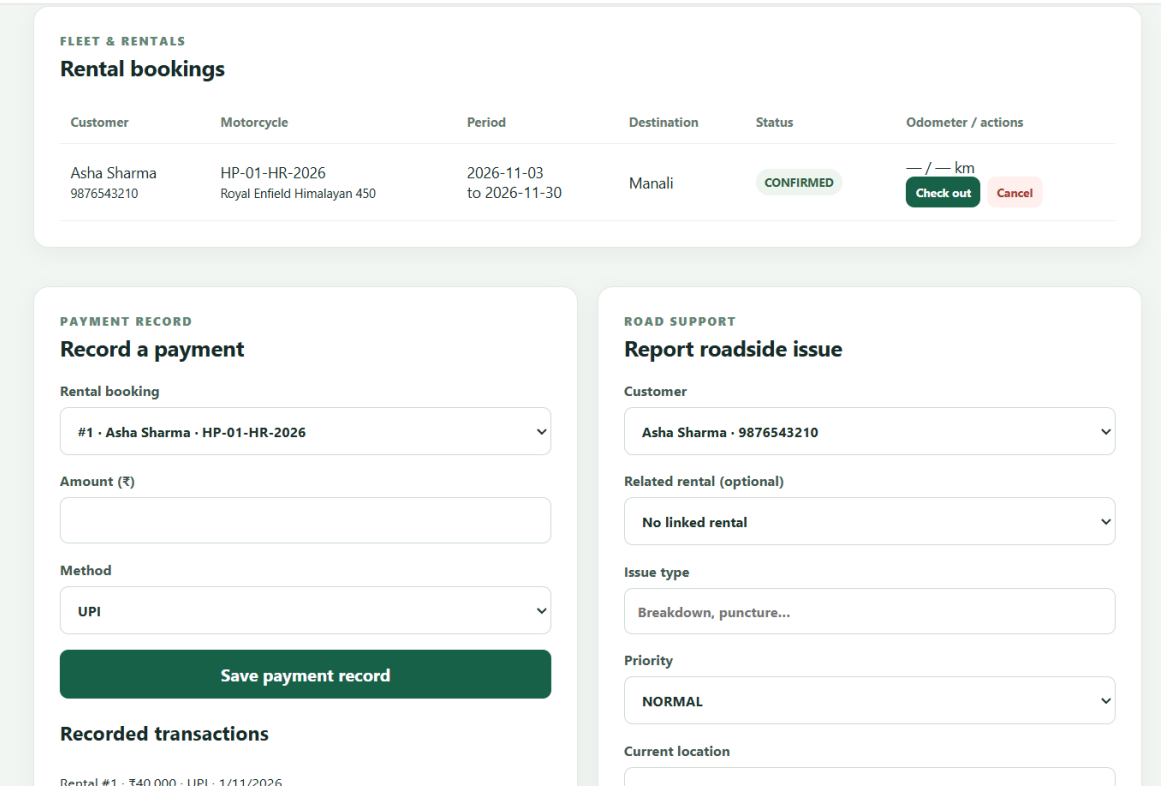


Figure 4: Confirmed booking, payment record form and roadside support form.

6. Conclusion and Limitations

Himalayan Ride and Care demonstrates how client-server communication, Java controllers, REST-style HTTP operations, JSON, Spring Boot, Hibernate/JPA and MySQL can support a small rental and assistance workflow. The local startup and database connection were confirmed in the supplied run output. The project is suitable as a teaching prototype and a base for further syllabus exercises.

Limitations: the prototype does not implement login, role-based permissions, explicit JSP pages, XML responses, live trip tracking, a payment provider, or automated dispatch. Current API methods use GET, POST and PATCH rather than implementing every method in the AIP exercise sheet. Future work could add authenticated roles, audit history, XML content negotiation, direct servlet/JSP demonstrations, tests and reporting by date or status.

References

[1] Advanced Internet Programming course syllabus supplied for this project.

[2] A Case Study on Himalayan Ride & Care, supplied project case study.

[3] Spring Boot Reference Documentation, 3.3 series. https://docs.spring.io/spring-boot/3.3/reference/

[4] Spring Data JPA Reference Documentation. https://docs.spring.io/spring-data/jpa/reference/

[5] MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[6] MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/